The effect of sensitive data presence on the attacker's dwell time and depth of exploration.

Navtej Soma, Santosh Sureshkumar, FNU Mahek, Zayd Mahfuz

Honeypot Chickens (group-a)

HACS202 (0101)

Bertrand Sobesto, Michel Cukier

December 07, 2025

Executive Summary (Zayd)

This experiment investigates whether increasing the amount of sensitive data within a high-interaction SSH honeypot leads attackers to stay longer and explore more deeply. Prior studies have shown that attackers tend to exhibit more persistent behavior when they perceive a system as valuable. Still, most research relied on low-interaction honeypots that cannot capture detailed behavioral traces. Our goal was to evaluate attacker behavior across four high-interaction containers configured with different levels of file sensitivity, using real-world attacks observed on public IP addresses.

Our research question is: Does increasing the amount of sensitive data in a system lead attackers to stay longer and explore more deeply compared to systems with mostly non-sensitive data? We hypothesize that attackers will exhibit longer session durations and execute more commands when the honeypot appears to contain sensitive data, reflecting increased interest and exploration. The null hypothesis states that file sensitivity has no impact on attacker behavior. To test this, we deployed four LXD containers exposed to the public internet: a control container with minimal non-sensitive files. And three treatment containers containing increasing levels of sensitive content. Each container was assigned a dedicated public IP address and configured with identical system settings to ensure that file sensitivity was the isolated treatment variable. We collected logs of every SSH connection, including the attacker's IP address, timestamps, session duration, and all executed commands.

Across the multi-week deployment, we observed thousands of attacks. Although most sessions were very short, likely the result of automated scanning, the number of meaningful interactions and the volume of commands executed were still sufficient for statistical analysis. The violin and box plots of attack durations showed heavily right-skewed distributions for every

configuration: Most of the sessions lasted only a few seconds, with only a small set of outliers remaining connected for much longer. Importantly, there were no statistically reliable differences in how long attackers stayed across configurations; even after excluding timeout sessions or restricting the analysis to unique IPs, the overall duration patterns remained similar between the control and all treatment containers.

In contrast, the number of commands executed showed a strong increasing trend aligned with file sensitivity. Attackers executed substantially more commands in Treatment 2 and Treatment 3 compared to the control container, suggesting deeper engagement and exploration, even if their total time spent did not significantly increase. After conducting statistical tests, we confirmed that command depth is the clearest indicator of behavioral change. In other words, attackers may not stay dramatically longer, but they do explore more when they believe valuable information is present.

Overall, our findings partially support our hypothesis: increased file sensitivity leads to deeper attack exploration but not reliably longer session durations. This suggests that attacker behavior is influenced by perceived data value, but the decision to stay connected may be constrained by automated tools, scanning workflows, or intrusion detection systems. Future research could explore more sophisticated deception strategies, such as dynamically changing file contents, adapting honeypot behavior in real time, or varying exposed credentials, to better identify which system characteristics most effectively influence and sustain attacker engagement.

Background Research (Zayd)

Honeypots play a central role in cybersecurity research because they provide controlled environments for studying attacker behavior without risking production systems. Early honeypot deployments were primarily low-interaction systems that offered limited insight, often capturing

only connection attempts or basic command activity. Modern high-interaction honeypots allow researchers to observe detailed post-compromise behavior, making them well-suited for examining how attackers explore and evaluate systems they successfully enter.

Research in cybersecurity shows that attacker behavior inside compromised systems is influenced by how valuable and realistic the system appears. Two of the most common metrics used to quantify such behavior are dwell time and depth of exploration. "Dwell time" refers to how long an attacker remains active within a system, while “depth of exploration" captures the extent to which an attacker navigates directories, opens files, or executes meaningful commands. These measures are widely used to evaluate how environmental characteristics, including file sensitivity, system configuration, and host credibility of stored data, shape attacker engagement.

Prior work has shown that the design and contents of an environment directly influence attacker interaction patterns. Qurbonaliyeva and Abduraxmanova (2025) investigated how different forms of bait affect attacker motivation and found that attackers invest more effort when the environment signals the presence of valuable information. This demonstrates that attackers are sensitive to contextual cues and reinforces the idea that variations in the sensitivity of stored data may meaningfully influence engagement. Our experiment applies this concept by systematically varying sensitive and non-sensitive files across containers to observe how it affects attacker behavior.

Similarly, Savage (2025) examined honeypots that used various deception strategies, showing that both dwell time and command activity varied depending on the environmental setup. While their work focused on deception mechanisms such as fake services or misleading system banners, the underlying takeaway, that system design shapes attacker decisions, supports our research approach. Rather than relying on deception, our study isolates the effect of data

sensitivity itself, allowing us to assess whether attackers respond to realistic file content without additional manipulation.

Understanding what attackers do after gaining access has been the subject of several experimental studies. Schneider et al. (2021), for example, analyzed over 200 real post-exploitation SSH sessions and found that attackers consistently engaged in directory traversal and file inspection when they believed the system contained valuable information. This highlights the depth of exploration as a critical behavioral metric and provides a meaningful foundation for analyzing command activity in our honeypots. Their findings also suggest that increases in exploration behavior can serve as a strong signal that attackers perceive value in the system's data.

Shah et al. (2020) studied attacker persistence across compromised enterprise accounts and found that adversaries often maintain access for extended periods, sometimes exploring slowly and following recognizable behavioral patterns rather than immediately exfiltrating data. This reinforces the relevance of dwell time as a research measure and highlights that attackers may exhibit meticulous decision-making in environments they perceive as legitimate or valuable. These insights justify our inclusion of both dwell time and exploration depth as primary metrics in evaluating how data sensitivity affects attacker behavior.

Collectively, this body of research demonstrates that attackers respond to environmental cues, particularly those related to system value and credibility, and that their behavior can be meaningfully measured using dwell time and depth of exploration. Building on this foundation, our study examines whether increasing the proportion of sensitive data in a high-interaction honeypot leads attackers to stay longer and explore more deeply. By isolating file sensitivity as

the key variable, we aim to contribute observed evidence to the broader understanding of how system design and data value shape adversarial engagement.

Experiment Design Changes

Initially, the model included three data categories (A, B, and C), each representing increasing levels of sensitive data, and four treatment levels specifying distinct proportions of these categories. While the design offered a strong theoretical foundation for studying attacker motivation, we realized during early development that it would be extremely difficult to execute and analyze in practice. Each proportional level required a sufficient number of attacker sessions to distinguish behavior between treatments, meaningfully, and the unpredictable frequency of real-world attacks made this infeasible. Because of these constraints, we re-evaluated the independent variable and ultimately simplified the model.

Rather than distributing attackers across multiple proportional mixes of sensitivity, we restructured the experiment to use a binary classification of data, sensitive and non-sensitive, resulting in four clear treatments: a control treatment with no files, one treatment with only non-sensitive data, a treatment with only sensitive data, and one with a mixture of both. We redefined our operational meaning of “sensitive” and “non-sensitive” data. Sensitive data was re-established as passwords, Social Security Numbers, financial information, and personally identifiable information (PII), while non-sensitive data included media files and generic documents. This ensured that our variables corresponded directly with attacker motivations. In addition, we removed the “number of files” variable, as it introduced unnecessary complexity without advancing the central hypothesis.

We also modified the timing of the recycling script. The experiment initially recycled containers every six hours, but this window resulted in too few attacker sessions being captured.

To increase the number of observations, the reset interval was progressively reduced to three hours, then thirty minutes, and finally ten minutes. This modification ensured that each honeypot was frequently restored to a clean state, maximizing exposure opportunities and improving the likelihood of obtaining meaningful data across all treatments.

We also corrected the randomization logic that determined which treatment was applied during each cycle. In earlier versions, the recycling script unintentionally favored Treatment 1 because of an error in how the random index was generated. As a result, some treatments received significantly more exposure than others, which undermined the validity of our comparisons. Once we identified the cause of the imbalance, we updated the randomization process to ensure that all four treatments were chosen with equal probability. This change was crucial for keeping the experiment fair, preventing biased attacker distribution, and ensuring that any variations in dwell time or exploration depth reflected genuine experimental effects rather than uneven deployment.

We also made several changes to improve the reliability of the automation pipeline and the quality of the data collected. In earlier versions, the recycling script depended on an active terminal session and would shut down if the connection to the LXC host was lost. This instability created a risk of data loss and disrupted the experiment's continuity. To solve this, we redesigned the script to run independently in the background, allowing continuous cycling and log collection over long durations. Additionally, the first rounds of attacker logs revealed issues such as incomplete Snoopy entries, inconsistent timestamp formatting, and partial command traces. These problems highlighted the importance of simplifying the experimental setup, as overly complex treatments would have amplified noise and increased preprocessing demands. By stabilizing the automation and refining how logs were collected and interpreted, we improved the

consistency of the dataset and ensured that the final design supported meaningful and statistically valid analysis.

Research Question / Hypothesis

Within the experiment conducted, the main purpose was to study the attack behavior and how much of an influence it had on the different types of treatments that were presented. The experiment focuses on the attacker engagement and exploration depth based on the time spent. This leads us to the research question of **how the presence of different types of decoy data affects attacker engagement and exploration depth time.** This question best summarizes the purpose of the experiment and is more helpful from the defensive side of cybersecurity strategy, as good assumptions and conclusions can be made about what attracts certain attackers and what deters them.

The main assumptions were that the treatments with the most sensitive data would have the most dwell time and attacker engagement in comparison to the less sensitive treatments. Most of the motivation for attackers stems from sensitive data, as it is generally deemed more valuable. Examples of the types of sensitive data it could contain include Social Security Numbers, license/passport information, banking details, etc. The attackers' main motivation closely correlates to a reward system where the higher sensitive data can lead to financial gain for the attackers while causing the most damage overall to the victim organization. Although accessing and searching through the sensitive information can be seen as more difficult, the attackers would be willing to take the risk as it follows the “high risk/high reward” protocol. With not much useful information being in the treatments with the less sensitive files, it can be inferred that these sensitive files would be left close to untouched, as the data from these files may not contain much useful information that can be beneficial for the attacker. This would

follow the “low risk/ low reward” protocol, as there is no real benefit nor risk for trying to go after this data. The treatments are divided into four treatments, where it follows the sequential structure of Treatment 3 (All sensitive files) > Treatment 2 (Mix of sensitive/non-sensitive) > Treatment 1 (all non-sensitive files) > Control (No files).

Null Hypothesis (H0): There is no significant correlation between the time spent or depth explored in comparison to the proportions of sensitive and non-sensitive data found. Overall, the level of sensitivity does not play a significant role in determining the attacker’s behavior. Essentially, all the treatments would be treated the same; all of these would be searched equally with no significant difference in attacker engagement or exploration depth time.

Alternate Hypothesis (H1): The higher the proportion or level of sensitive data there is in a treatment, the more significant the upward correlation is seen with the exploration time among attackers in comparison with low sensitive data. Attackers will spend more time traversing through directories and files to look for highly sensitive information.

Experiment Design

The experiment was designed to evaluate how the type and sensitivity of data stored within a system influence attacker behavior. To study this relationship in a controlled and repeatable way, we built a fully automated honeypot infrastructure using four LXC containers running on a dedicated host machine. Each container was configured to behave like a normal Linux system that an attacker might encounter in the real world. The only difference between the four honeypots (treatment levels) was the data they contained; some held only sensitive files, some held only non-sensitive files, one contained a mixture of both, and one acted as a control with no meaningful files at all. Every other system characteristic, including the operating system version, installed packages, SSH configuration, user accounts, network exposure, and logging

tools, was kept consistent across all containers. For networking, we used LXC's default bridged setup so that each honeypot received its own internal IP address and could be reached from the public internet.

Our experiment followed a repeating lifecycle where each honeypot was deployed, exposed to the internet, monitored, logged, and then reset before the next cycle began. The system was built around three core elements:

- LXC Host: Served as the environment for running and managing containers, snapshots, and automation processes.
- Four treatment containers:
    - Control: Container with no meaningful files. This served as a baseline for attacker behavior.
    - Treatment 1: Container with only non-sensitive files such as media files, PDFs. This served as a low-value content scenario.
    - Treatment 2: Container with only sensitive files such as password lists, SSNs, and PII-like documents. This was a high-value target simulation
    - Treatment 3: Container with a mixed combination of sensitive and non-sensitive files. This served as a realistic system representation.
- Recycling script: Handled treatment selection, honeypot restoration, intrusion checks, log collection, timing, and continuous looping without manual involvement.

For the container setup, we created four base honeypots using the same Linux image so that every environment was identical except for the data inside it. All containers used Snoopy for command monitoring. After configuring each container, we took a clean snapshot so the

recycling script could restore the same starting point every time. This ensured that attackers always interacted with a fresh system and prevented any changes from carrying over between cycles.

To automate the experiment, we wrote a recycling script that handled every stage of the honeypot lifecycle from start to finish. The script randomly selected one of the four treatments, restored the corresponding snapshot into a new container, and launched it so it appeared online and ready for attacker activity. Once the honeypot was running, the script controlled the runtime window, checked authentication logs for successful logins, terminated any active SSH sessions at the end of the cycle, and collected all relevant logs, including auth logs, Snoopy command logs, system logs, and basic metrics. After saving the logs, the script deleted the container entirely and immediately began preparing the next cycle using the same process.

For storage, we kept everything on the LXC host, so all logs and cycle outputs were easy to manage and consistently organized. Each cycle generated a timestamped folder containing the raw logs, metrics, and container information. Keeping every cycle separate made it much easier to trace attacker activity back to the exact treatment and deployment it came from. The base treatment containers and their snapshots also stayed on the host to ensure that each cycle restored the same initial state.

To ensure each treatment had a fair chance of being deployed, we used a simple randomization strategy inside the automation script. At the start of every cycle, the script generated a random index and selected one of the four treatments. After fixing an early issue in the randomization logic, each treatment was selected with equal probability, which kept the experiment fair and allowed us to compare attacker behavior across treatments without bias.

Data Collection

Our data collection involved the continuous deployment of four different honeypot configurations designed to investigate whether the sensitivity level of files inside a honeypot influences attacker behavior. Throughout this period, our container-based honeypot infrastructure automatically captured all attacker interactions in real time. Because all data was generated and logged automatically, the dataset reflects unbiased attacker behavior without manual oversight or interruption.

Across all deployments, we collected several categories of information necessary to characterize attacker sessions and measure behavioral differences. First, we recorded detailed session metadata, including source IP addresses, unique session identifiers, and precise timestamps marking the beginning and end of each attack. These timestamps were used to calculate total session durations in milliseconds. Second, we collected all command execution data, including both interactive and non-interactive commands. This allowed us to measure the depth of attacker activity based on the total number of commands executed. Third, we captured temporal records for every event, ensuring that each action could be placed within a consistent timeline of interaction. Finally, for each attack, we recorded the specific honeypot configuration, Control, Treatment 1, Treatment 2, or Treatment 3, to have comparisons across different sensitivity levels.

All of these logs, SSH logs, command logs, system logs, and timestamp logs, were automatically streamed into centralized CSV files. By the end of the deployment period, we had collected a total of **30,080 attacks** spread across the four configurations: **6,963 in Control**, 1**0,484 in Treatment 1**, **6,543 in Treatment 2**, and **6,090 in Treatment 3**. In addition, we

identified **11,748 unique IP addresses**, each representing at least one recorded attack. These tables can be referenced in Appendix B.

A substantial amount of data preprocessing was required before any analysis could take place. Several data processing decisions were necessary to ensure that the dataset accurately reflected attacker behavior and did not contain system-generated artifacts. One of the primary preprocessing tasks involved handling timeout sessions. Because some attackers remained connected until reaching the honeypot's maximum allowed session duration, these timeout sessions were excluded from any duration-based analysis to prevent artificially inflated values. However, we retained them for command-count analysis since the number of executed commands remains valid regardless of session length.

Next, we grouped and categorized the dataset to answer the research question from multiple perspectives. We created subsets that included all attacks, only the first attack from each unique IP address, and configuration-specific groups. Grouping the data this way enabled us to determine whether results were being driven by a small number of highly active attackers or whether patterns were consistent across the entire population.

After preprocessing, the final dataset sizes were as follows: approximately **29,500 attacks** for duration analysis (after removing timeouts and negative durations), **30,080 attacks** for command-count analysis (full dataset), and **11,748 unique IP entries** for unique-attacker analysis.

A standardized command-counting methodology was applied to every session. For each attack, the number of commands was defined as the sum of interactive and non-interactive commands, each separated by semicolons. This approach captures the complexity and breadth of attacker exploration within each honeypot.

Before conducting any statistical analysis, we established an interpretation framework based on theoretical expectations about attacker behavior. Duration was treated as a measure of exploration time, how long an attacker remained engaged inside the honeypot, while the number of commands served as a proxy for exploration depth. By comparing the distributions of these metrics across different honeypot configurations, we aimed to assess whether the sensitivity of the files influenced attacker engagement. This framework guided how we prepared the data, but did not involve any actual analysis, which is reserved for the following section.

Data Analysis

After completing data preprocessing, we analyzed the dataset using both parametric and non-parametric statistical methods to evaluate whether honeypot sensitivity affected attacker behavior. Two dependent variables, session duration and command count, served as the primary behavioral indicators. Because our study design involved four distinct honeypot configurations, we employed statistical tests capable of comparing multiple groups simultaneously.

For parametric testing, we used one-way ANOVA to determine whether the mean duration or the mean command count differed significantly across the four configurations. ANOVA was appropriate because it evaluates variance both within and between groups and provides an overall significance test before conducting any pairwise comparisons. When ANOVA produced a p-value below our significance threshold of 0.1, we followed up with Tukey's Honestly Significant Difference test to identify which specific group pairs differed.

Since preliminary visualizations suggested non-normal distributions, particularly for command counts, we also employed the Kruskal–Wallis test, a non-parametric method that compares the median ranks across groups without relying on normal distribution assumptions.

Whenever Kruskal–Wallis indicated significant differences, we conducted Dunn's post-hoc test to determine the specific pairs that differed.

For **attack duration (all attacks excluding timeouts)**, ANOVA results were significant ($p < 0.0001$), indicating that durations varied across configurations. Tukey's post-hoc results revealed that Treatment 1 consistently recorded significantly longer durations than Control, Treatment 2, and Treatment 3, while the remaining comparisons did not show meaningful differences. Kruskal–Wallis results ($p < 0.0001$) supported these findings, and Dunn's test again identified Treatment 1 as the configuration that differed from the others. However, this pattern did not follow the predicted trend of increasing duration with increasing sensitivity. Because the differences were not aligned with our research hypothesis, **we cannot reject the null hypothesis for attack duration**.

When examining **only the first attack from each unique IP address**, the same pattern appeared. Treatment 1 again showed the longest durations, and both ANOVA and Kruskal–Wallis returned significant results. Tukey and Dunn's tests confirmed significant differences between Treatment 1 and the other configurations. Since the unexpected pattern persisted even after controlling for repeat attackers, we again conclude that **the null hypothesis for duration cannot be rejected**.

The analysis of **command count** produced a very different outcome. For the full dataset, ANOVA indicated highly significant differences across all four configurations ($p < 0.0001$). Tukey's test showed that **every pairwise comparison** was statistically significant. Moreover, the differences followed a clear and monotonic progression: attackers in control executed fewer commands than in Treatment 1, fewer than in Treatment 2, and far fewer than in Treatment 3. Kruskal–Wallis results mirrored this outcome ($p < 0.0001$), and Dunn's test again confirmed

widespread pairwise differences, with command counts increasing consistently alongside sensitivity levels.

When restricting the dataset to **unique IP addresses only**, we observed the exact same pattern. All statistical tests remained significant, and the increase in command execution from control to Treatment 3 persisted. These results show that the trend is not driven by repeated attackers but instead reflects genuine, widespread behavioral differences.

Based on the command-count analyses, we conclude that **the null hypothesis can be rejected** for command execution behavior. Attackers clearly executed more commands as the sensitivity of the files increased, showing a strong sensitivity-dependent effect.

Taken together, our statistical findings reveal a divided outcome. For duration, the data shows statistically significant differences across configurations, but the differences follow an unexpected pattern and do not support the hypothesis that attackers stay longer when more sensitive data is present. Therefore, we cannot reject the null hypothesis for duration. In contrast, command count consistently and strongly increases with sensitivity level across all datasets and all statistical methods. This provides compelling evidence that sensitive data encourages attackers to explore more deeply, even if it does not necessarily cause them to remain connected for longer periods.

Conclusion

The purpose of this study was to determine whether the sensitivity of files placed within a honeypot influences attacker behavior. By deploying four distinct honeypot configurations and analyzing both the depth and duration of attacker interactions, we sought to understand whether attackers respond differently when presented with increasingly sensitive data. Our findings show a clear divergence between behavioral dimensions: while time spent in the honeypot did not

reliably track with sensitivity level, the number of commands executed, a strong proxy for exploration depth, displayed a consistent and meaningful upward trend as sensitivity increased.

The most robust and interpretable result from this work is the discovery of a **dose-response relationship between data sensitivity and command execution**. Across all attacks and within the subset of unique IP addresses, attackers executed progressively more commands from the control configuration through Treatment 1, Treatment 2, and ultimately Treatment 3. This pattern was reinforced by both parametric and non-parametric statistical tests, all of which confirmed significant pairwise differences. These findings indicate that attackers do not merely notice the presence of more sensitive files; they actively engage more deeply when such files are present. This suggests that sensitive content alters attacker decision-making, likely increasing curiosity, perceived value, or the motivation to enumerate and exploit the environment.

In contrast, the analysis of **session duration** produced statistically significant differences across configurations but did not follow the predicted ordering of sensitivity levels. Treatment 1 consistently showed longer durations, whereas higher-sensitivity configurations did not exhibit the extended engagement we anticipated. This outcome highlights that time alone may be a poor indicator of attacker interest, as it can be influenced by methodological differences, automated scripts, and variation in attacker objectives. Taken together, the results suggest that **command count is a more reliable measure of attacker intent** than raw session duration.

Overall, this study demonstrates that attackers respond behaviorally to the sensitivity of data they encounter, but in ways that are more nuanced than simply staying connected longer. The consistent increases in command execution across sensitivity levels provide strong evidence that attackers probe more thoroughly when they believe the environment may contain valuable

assets. This finding supports the broader idea that honeypot design, and specifically, the type of bait provided, can meaningfully shape attacker behavior.

If this line of research were extended, several promising directions would emerge. First, future work could explore **which types of sensitive files most effectively influence attacker decisions**, such as financial documents, credentials, source code, or system configurations. Understanding which categories of files are most persuasive could improve the design of deception technologies. Second, a natural extension would be to examine **how different attacker profiles, such as automated bots, opportunistic attackers, or targeted adversaries, respond differently to varying data sensitivities**. Distinguishing these behavioral classes could allow defenders to tailor honeypot environments strategically. Third, it would be valuable to analyze **how attackers transition between actions**, for example, whether certain file types trigger privilege escalation attempts, lateral movement simulation, or reconnaissance patterns. Studying these behavioral pathways could help map attacker workflows and support the development of predictive threat models.

Finally, the work could be enhanced by exploring **adaptive honeypots**, where the environment dynamically changes based on attacker actions. Such a system could, for instance, reveal additional sensitive files as attackers demonstrate increased interest, allowing researchers to observe decision-making processes in richer detail. This direction aligns with a growing interest in active cyber deception, where environments are designed to learn from attackers in real time.

In conclusion, this research provides clear evidence that the presence of sensitive data influences attacker engagement, particularly in terms of exploration depth. While session duration did not behave as initially hypothesized, the consistent escalation of command

execution across sensitivity levels offers meaningful insight into attacker motivations and interactions with deceptive environments. These findings reinforce the value of strategically designed honeypots and open the door to more sophisticated deception-driven research in cybersecurity.

Appendix A

The biggest surprise was the number of attacks that were logged based on how many the treatments received. Over the course of a few weeks, a total of 30,080 attacks were logged. Out of those attacks, there was a total record of 11,748 unique IPs across the four treatments. These attacks took place as soon as the containers were active, which highlights how significant global scanning is. A bimodal distribution of attack durations was discovered as well, which showed that most of the attackers were automated bots that disconnected within seconds of entering the environment; however, some stayed until the recycling limit. This pattern remained the same among all the different treatments, indicating that most of these attacks were automated and were not driven by the different types of data in the honeypot. One of the most unexpected results was that the attackers spent more time in Treatment 1 than in any other treatment. Treatment 1 is supposed to contain only non-sensitive files, which goes completely against the hypothesis originated. This overall showed that there was no real interest in any of the files. For common execution, however, the attackers ran more commands towards the more sensitive treatments in an attempt to gain access.

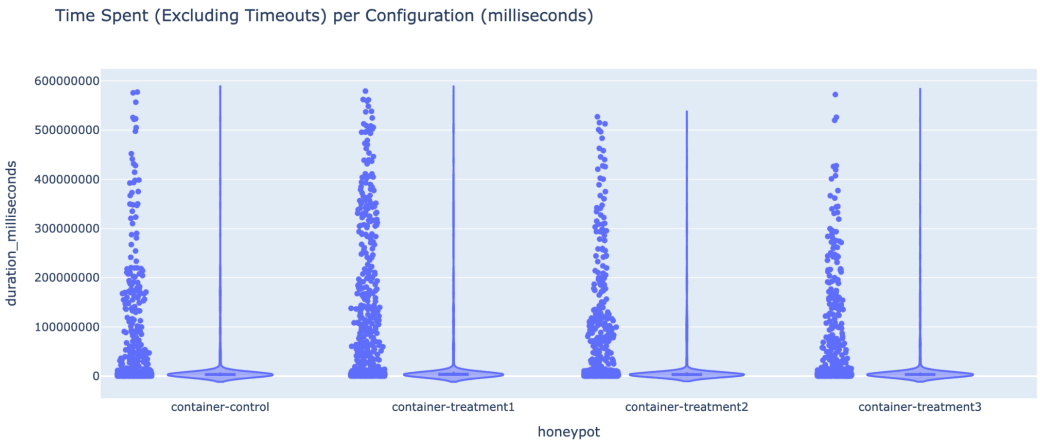
The early issues through this experiment were mainly a flaw in the randomization logic, which gave Treatment 1 more priority and exposure over the others. The entire randomization part of the script had to be rewritten and tested to make sure the process was correct. The recycling interval was a major setback as well, as the initial interval of 6 hours yielded no data.

With the suggestions of the instructor, the interval was reduced to every 30 minutes, which was then finalized at 10 minutes. Although this increased data volume, the attackers were essentially cut short, which forced the exclusion of the data from that analysis. Other minor issues that were faced included incomplete Snoopy logs and inconsistent time stamps, which did not affect the overall experiment to make a significant difference.

This project gave us a real-world example of working with real attacker data and helped us produce strong statistical results. Studying the behavior of the attackers gave our group a practical approach to how an attacker in real time thinks and what actions they perform. However, setting up this experiment was quite difficult as our group had only a little experience with Linux. Although we were able to understand it quite fast, this may have delayed the experiment by a small amount of time due to simple mistakes. Having the timeline compressed to only a month made it harder to perform trial and error with our experiments and forced us to set it up and collect data quickly. Despite all of these setbacks, everything worked towards the end, and we were able to get solid results to come to conclusions on this experiment.

Appendix B

- Number of attacks per IP address
  - Table
- Number of attacks per HP configuration
  - Table
- Data + violin plot of **all attack durations per HP configuration** (when **EXCLUDING** the attacks that reach HP time limit)



- ANOVA test (sm.stats.anova_lm) and apply a Tukey HSD test (pairwise_tukeyhsd) **Time Spent per Configuration**

```
================================================================================
TEST 1: ANOVA - Time Spent per Configuration (Excluding Timeouts, ms)
================================================================================
                    sum_sq      df          F        PR(>F)
C(honeypot)   2.785789e+17     3.0  17.555483  2.382796e-11
Residual      3.571466e+19  6752.0        NaN           NaN

P-value: 0.0000
✓ ANOVA SIGNIFICANT (p=0.0000 < 0.1)
/tmp/ipython-input-14064394.py:11: FutureWarning:
```

```
================================================================================
TEST : TUKEY HSD POST-HOC (Time Durations, No Timeouts)
================================================================================
                    Multiple Comparison of Means - Tukey HSD, FWER=0.10
===================================================================================================
       group1              group2          meandiff   p-adj     lower        upper    reject
---------------------------------------------------------------------------------------------------
   container-control container-treatment1 -226847.6139 0.0004 -356480.1203 -97215.1074   True
   container-control container-treatment2   64594.7261 0.6914  -70350.5582 199540.0105  False
   container-control container-treatment3   23531.7009 0.9791 -112938.9549 160002.3566  False
container-treatment1 container-treatment2    291442.34    0.0  161987.6912 420896.9888   True
container-treatment1 container-treatment3  250379.3147 0.0001  119335.3671 381423.2623   True
container-treatment2 container-treatment3  -41063.0253 0.9008 -177364.7466  95238.6961  False
---------------------------------------------------------------------------------------------------
```

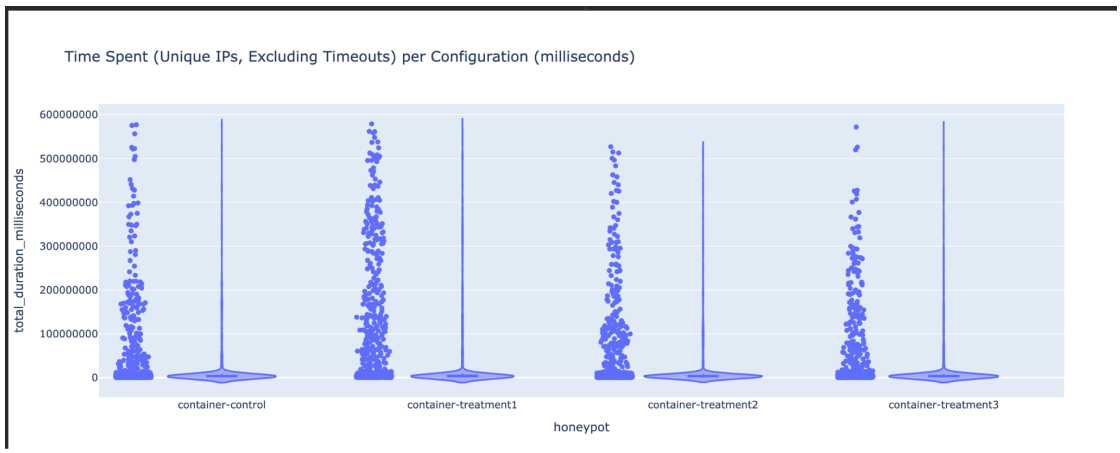
- Kruskall-Wallis test (sp.stats.kruskal) and a Dunn test (posthoc_dunn) for **Time Spent per Configuration**

```
================================================================================
TEST : KRUSKAL-WALLIS - Time Spent per Configuration (Time Durations, No Timeouts)
================================================================================
H-statistic: 53.4336
P-value: 0.0000

✓ KRUSKAL-WALLIS SIGNIFICANT (p=0.0000 < 0.1)
→ Significant difference in attack duration distributions across honeypots
```

```
================================================================================
TEST: DUNN POST-HOC (Time Durations, No Timeouts)
================================================================================
                      container-control  container-treatment1  \
container-control          1.000000e+00          4.370392e-07
container-treatment1       4.370392e-07          1.000000e+00
container-treatment2       1.000000e+00          2.829564e-10
container-treatment3       1.000000e+00          1.296699e-06

                      container-treatment2  container-treatment3
container-control              1.000000e+00              1.000000
container-treatment1           2.829564e-10              0.000001
container-treatment2           1.000000e+00              1.000000
container-treatment3           1.000000e+00              1.000000
```

- Interpretation: Values < 0.1 indicate significant pairwise differences

- Data + violin of attack durations for **unique IP addresses** (first attack from that IP address) per HP configuration



- ANOVA test (sm.stats.anova_lm) and apply a Tukey HSD test (pairwise_tukeyhsd) for **Time Spent for Unique IPs excluding timeouts**

```
================================================================================
ANOVA TEST: Time Spent (Unique IPs, Excluding Timeouts)
================================================================================
                    sum_sq     df         F        PR(>F)
C(honeypot)   3.769478e+08    3.0  42.24722  2.619430e-25
Residual      2.325780e+09  782.0       NaN           NaN

P-value: 0.0000
✓ ANOVA SIGNIFICANT (p=0.0000 < 0.1)
/tmp/ipython-input-1133546675.py:10: FutureWarning:

Series.__getitem__ treating keys as positions is deprecated. In a future version,
```

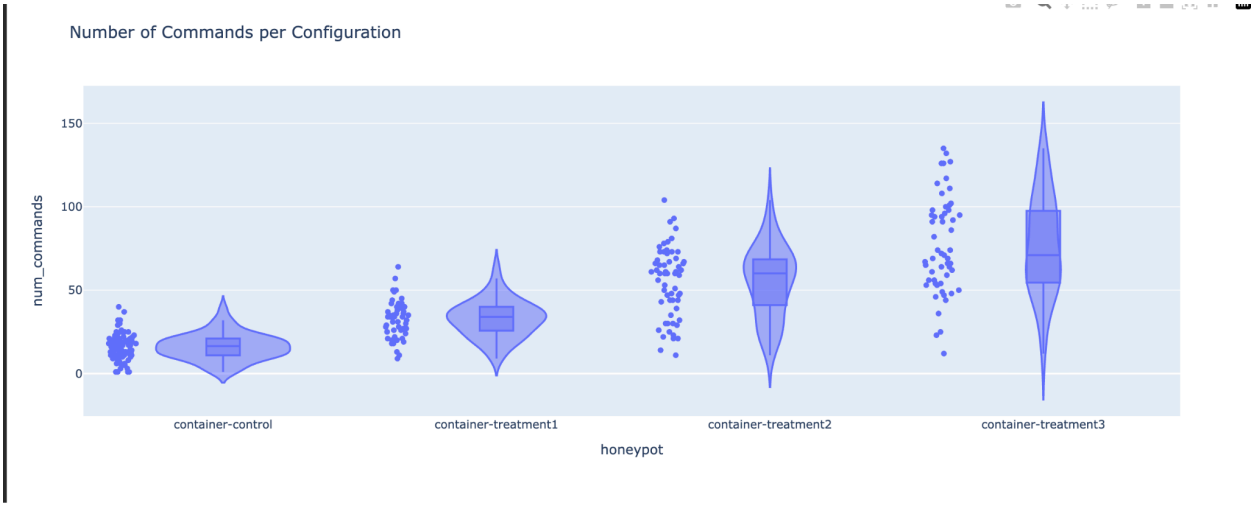
```
================================================================================
TUKEY HSD POST-HOC: Time Spent (Unique IPs, Excluding Timeouts)
================================================================================
                 Multiple Comparison of Means - Tukey HSD, FWER=0.10
================================================================================
       group1              group2         meandiff  p-adj    lower      upper    reject
--------------------------------------------------------------------------------------
   container-control container-treatment1 -1417.1225    0.0 -1782.7202 -1051.5249   True
   container-control container-treatment2  -248.2122 0.5677  -688.8494    192.425  False
   container-control container-treatment3   178.0351 0.8522  -329.6651   685.7354  False
container-treatment1 container-treatment2  1168.9103    0.0   787.2732  1550.5475   True
container-treatment1 container-treatment3  1595.1577    0.0  1137.7243   2052.591   True
container-treatment2 container-treatment3   426.2473 0.2359   -93.1223   945.6169  False
--------------------------------------------------------------------------------------
```

- Kruskall-Wallis test (sp.stats.kruskal) and apply a Dunn test (posthoc_dunn) for **Time Spent for Unique IPs excluding timeouts**

```
================================================================================
KRUSKAL-WALLIS TEST: Time Spent (Unique IPs, Excluding Timeouts)
================================================================================
H-statistic: 154.7184
P-value: 0.0000

✓ KRUSKAL-WALLIS SIGNIFICANT (p=0.0000 < 0.1)
```

```
...
================================================================================
DUNN POST-HOC: Time Spent (Unique IPs, Excluding Timeouts)
================================================================================
                     container-control  container-treatment1  \
container-control         1.000000e+00          2.969707e-23
container-treatment1      2.969707e-23          1.000000e+00
container-treatment2      1.000000e+00          1.599037e-15
container-treatment3      1.000000e+00          2.226741e-15

                     container-treatment2  container-treatment3
container-control             1.000000e+00          1.000000e+00
container-treatment1          1.599037e-15          2.226741e-15
container-treatment2          1.000000e+00          1.000000e+00
container-treatment3          1.000000e+00          1.000000e+00

Interpretation: Values < 0.1 indicate significant pairwise differences
```

- Data + violin plot of the **number of commands during each attack per HP configuration**



- ANOVA test (sm.stats.anova_lm) and a Tukey HSD test (pairwise_tukeyhsd) for **Commands per Configuration**

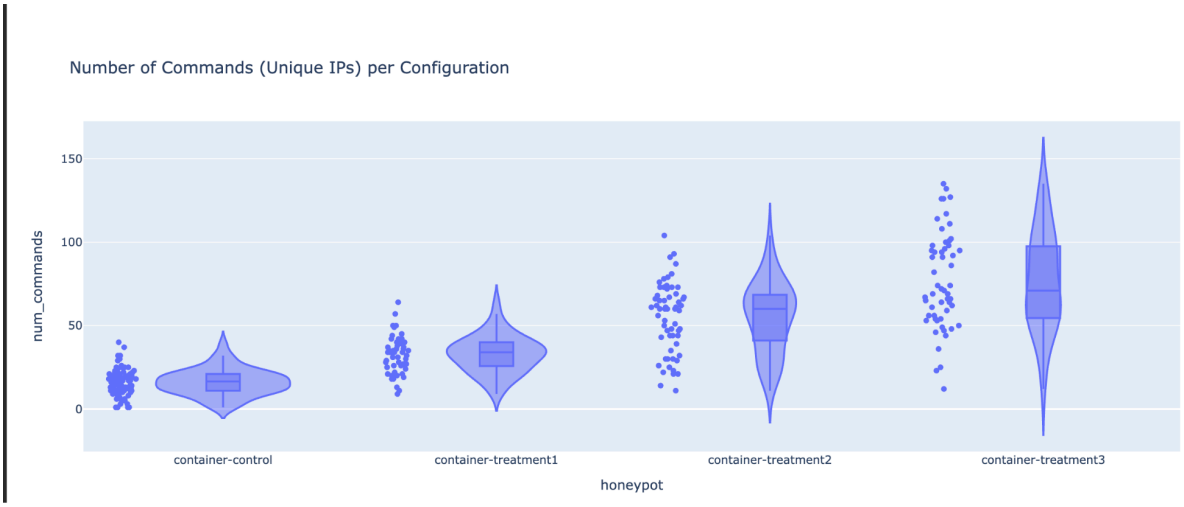
```
================================================================================
ANOVA: Number of Commands During Each Attack per HP Configuration
================================================================================
                    sum_sq     df           F        PR(>F)
C(honeypot)  139981.374155    3.0  140.555333  1.569115e-53
Residual      83656.985220  252.0         NaN           NaN

P-value: 0.0000
✓ ANOVA SIGNIFICANT (p=0.0000 < 0.1)

================================================================================
TUKEY HSD POST-HOC: Number of Commands During Each Attack
================================================================================
             Multiple Comparison of Means - Tukey HSD, FWER=0.10
================================================================================
       group1              group2        meandiff p-adj  lower   upper  reject
--------------------------------------------------------------------------------
   container-control container-treatment1  16.6677   0.0  9.3711 23.9642   True
   container-control container-treatment2  38.8152   0.0 31.7892 45.8411   True
   container-control container-treatment3  60.6591   0.0 53.4458 67.8724   True
container-treatment1 container-treatment2  22.1475   0.0 14.2368 30.0582   True
container-treatment1 container-treatment3  43.9914   0.0 35.9138  52.069   True
container-treatment2 container-treatment3  21.8439   0.0   14.01 29.6779   True
--------------------------------------------------------------------------------
```

- Kruskall-Wallis test (sp.stats.kruskal) and a Dunn test (posthoc_dunn) for **Commands per Configuration**

```
================================================================================
TEST 4: KRUSKAL-WALLIS - Commands per Configuration (All Data)
================================================================================
H-statistic: 172.4929
P-value: 0.0000

✓ KRUSKAL-WALLIS SIGNIFICANT (p=0.0000 < 0.1)
→ Significant difference in command distributions
```

```
...
================================================================================
TEST 5: DUNN POST-HOC (All Commands)
================================================================================
                      container-control  container-treatment1  \
container-control          1.000000e+00          1.502889e-06
container-treatment1       1.502889e-06          1.000000e+00
container-treatment2       4.236250e-21          9.384294e-04
container-treatment3       2.804387e-32          9.416999e-09

                      container-treatment2  container-treatment3
container-control              4.236250e-21          2.804387e-32
container-treatment1           9.384294e-04          9.416999e-09
container-treatment2           1.000000e+00          9.647896e-02
container-treatment3           9.647896e-02          1.000000e+00

Interpretation: Values < 0.1 indicate significant pairwise differences
```

- Data + violin plot for **the number of commands during each attack for unique IP addresses** (first attack from that IP address) per HP configuration



- ANOVA test (sm.stats.anova_lm) and a Tukey HSD test (pairwise_tukeyhsd) for for **Number of Commands per Configuration for Unique IPs**

```
================================================================================
ANOVA: Number of Commands per Configuration (Unique IPs)
================================================================================
                    sum_sq     df           F        PR(>F)
C(honeypot)  139981.374155    3.0  140.555333  1.569115e-53
Residual      83656.985220  252.0         NaN           NaN

P-value: 0.0000
✓ ANOVA SIGNIFICANT (p=0.0000 < 0.1)

================================================================================
TUKEY HSD POST-HOC: Number of Commands (Unique IPs)
================================================================================
              Multiple Comparison of Means - Tukey HSD, FWER=0.10
================================================================================
       group1              group2        meandiff p-adj  lower   upper  reject
--------------------------------------------------------------------------------
   container-control container-treatment1  16.6677   0.0  9.3711 23.9642   True
   container-control container-treatment2  38.8152   0.0 31.7892 45.8411   True
   container-control container-treatment3  60.6591   0.0 53.4458 67.8724   True
container-treatment1 container-treatment2  22.1475   0.0 14.2368 30.0582   True
container-treatment1 container-treatment3  43.9914   0.0 35.9138  52.069   True
container-treatment2 container-treatment3  21.8439   0.0   14.01 29.6779   True
--------------------------------------------------------------------------------
```

- Kruskall-Wallis test (sp.stats.kruskal) and a Dunn test (posthoc_dunn) for **Number of Commands per Configuration for Unique IPs**

```
================================================================================
KRUSKAL-WALLIS: Number of Commands per Configuration (Unique IPs)
================================================================================
H-statistic: 172.4929
P-value: 0.0000

✓ KRUSKAL-WALLIS SIGNIFICANT (p=0.0000 < 0.1)
→ Significant difference in command distributions (unique IPs) across honeypots

================================================================================
DUNN POST-HOC: Number of Commands (Unique IPs)
================================================================================
                      container-control  container-treatment1  \
container-control          1.000000e+00          1.502889e-06
container-treatment1       1.502889e-06          1.000000e+00
container-treatment2       4.236250e-21          9.384294e-04
container-treatment3       2.804387e-32          9.416999e-09

                      container-treatment2  container-treatment3
container-control              4.236250e-21          2.804387e-32
container-treatment1           9.384294e-04          9.416999e-09
container-treatment2           1.000000e+00          9.647896e-02
container-treatment3           9.647896e-02          1.000000e+00

Interpretation: Values < 0.1 indicate significant pairwise differences
```

Appendix C

Recycling Script:

```
#!/bin/bash

# Honeypot Recycling Script

# Automatically cycles through treatment levels for data collection

# Configuration

HONEYPOT_LIFETIME_MINUTES=10

LOG_STORAGE_DIR="/root/honeypot_logs"

ACTIVE_CONTAINER="honeypot-active"

# Treatment configurations (container_name:snapshot_name)

TREATMENTS=(

    "container-control:first-control-snapshot"
```

```
    "container-treatment1:first-treatment1-snapshot"
    "container-treatment2:first-treatment2-snapshot"
    "container-treatment3:first-treatment3-snapshot"
)
# Create log storage directory
mkdir -p "${LOG_STORAGE_DIR}"
# Function to get random treatment
get_random_treatment() {
    local random_index=$((RANDOM % ${#TREATMENTS[@]}))
    echo "${TREATMENTS[$random_index]}"
}
# Function to collect logs from container
collect_logs() {
    local container=$1
    local timestamp=$(date '+%Y%m%d_%H%M%S')
    local log_dir="${LOG_STORAGE_DIR}/${container}_${timestamp}"
    echo "[$(date)] Collecting logs from ${container}..."
    mkdir -p "${log_dir}"
    # Copy auth.log (contains Snoopy data and SSH logins)
    lxc file pull "${container}/var/log/auth.log" "${log_dir}/auth.log" 2>/dev/null || echo "No
auth.log found"
    # Copy syslog if it exists
```

```
    lxc file pull "${container}/var/log/syslog" "${log_dir}/syslog" 2>/dev/null || echo "No syslog
found"
    # Save container info
    echo "Container: ${container}" > "${log_dir}/container_info.txt"
    echo "Timestamp: ${timestamp}" >> "${log_dir}/container_info.txt"
    echo "Treatment: ${container}" >> "${log_dir}/container_info.txt”
    # Calculate basic metrics
    calculate_metrics "${container}" "${log_dir}"
    echo "[$(date)] Logs saved to ${log_dir}"
}
# Function to calculate metrics
calculate_metrics() {
    local container=$1
    local log_dir=$2
    local metrics_file="${log_dir}/metrics.txt"

    echo "=== HONEYPOT METRICS ===" > "${metrics_file}"
    echo "Container: ${container}" >> "${metrics_file}"
    echo "Collection Time: $(date)" >> "${metrics_file}"
    echo "" >> "${metrics_file}"
    # Count SSH login attempts
    local login_attempts=$(grep -c "sshd.*Failed password" "${log_dir}/auth.log" 2>/dev/null ||
echo "0")
```

```
    echo "Failed SSH attempts: ${login_attempts}" >> "${metrics_file}"
    # Count successful logins
    local successful_logins=$(grep -c "sshd.*Accepted password" "${log_dir}/auth.log"
2>/dev/null || echo "0")
    echo "Successful SSH logins: ${successful_logins}" >> "${metrics_file}"
    # Count total commands executed (Snoopy logs)
    local total_commands=$(grep -c "snoopy\[" "${log_dir}/auth.log" 2>/dev/null || echo "0")
    echo "Total commands executed: ${total_commands}" >> "${metrics_file}"
    # Extract unique commands
    grep "snoopy\[" "${log_dir}/auth.log" 2>/dev/null | \
        sed -n 's/.*filename:\([^]]*\)].*/\1/p' | \
        sort | uniq -c | sort -rn > "${log_dir}/unique_commands.txt"

    echo "" >> "${metrics_file}"
    echo "See unique_commands.txt for command breakdown" >> "${metrics_file}"
}
# Function to terminate active SSH sessions
terminate_ssh_sessions() {
    local container=$1
    echo "[$(date)] Terminating SSH sessions in ${container}..."
    lxc exec "${container}" -- pkill -9 sshd 2>/dev/null || true
    sleep 2
}
```

```
# Function to deploy a honeypot
deploy_honeypot() {
    local treatment=$1
    local container_name=$(echo "$treatment" | cut -d: -f1)
    local snapshot_name=$(echo "$treatment" | cut -d: -f2)
    echo "[$(date)] Deploying honeypot: ${container_name} from snapshot ${snapshot_name}"
    # Delete active container if it exists
    if lxc info "${ACTIVE_CONTAINER}" &>/dev/null; then
        echo "[$(date)] Removing old active container..."
        lxc stop "${ACTIVE_CONTAINER}" --force 2>/dev/null || true
        lxc delete "${ACTIVE_CONTAINER}" --force 2>/dev/null || true
    fi
    echo "[$(date)] Restoring ${container_name}/${snapshot_name}..."
    lxc copy "${container_name}/${snapshot_name}" "${ACTIVE_CONTAINER}" --profile
profile-active
    # Start the container
    echo "[$(date)] Starting container..."
    lxc start "${ACTIVE_CONTAINER}"
    # Wait for container to be ready
    sleep 5
    # Get and display the assigned IP
    local container_ip=$(lxc list "${ACTIVE_CONTAINER}" -c 4 --format csv | cut -d' ' -f1)
    echo "[$(date)] Container IP: ${container_ip}"
```

```
    # Record deployment
    echo "[$(date)] Honeypot deployed: ${container_name}"
    echo "${container_name}" > /tmp/current_treatment.txt
}

# Function to check if container has been accessed
check_for_intrusion() {
    local container=$1
    local auth_log=$(lxc exec "${container}" -- tail -100 /var/log/auth.log 2>/dev/null || echo "")
    # Check for successful SSH login
    if echo "$auth_log" | grep -q "sshd.*Accepted password"; then
        return 0  # Intrusion detected
    fi
    return 1  # No intrusion
}
# Main recycling loop
recycle_loop() {
    local cycle_count=0
    echo "============================================="
    echo "Honeypot Recycling Script Started"
    echo "Lifetime per deployment: ${HONEYPOT_LIFETIME_MINUTES} minutes"
    echo "Log directory: ${LOG_STORAGE_DIR}"
    echo "============================================="
```

```
while true; do
    cycle_count=$((cycle_count + 1))
    echo ""
    echo "[$(date)] ===== CYCLE ${cycle_count} ====="
    # Select random treatment
    treatment=$(get_random_treatment)
    container_name=$(echo "$treatment" | cut -d: -f1)
    echo "[$(date)] Selected treatment: ${container_name}"
    # Deploy honeypot
    deploy_honeypot "${treatment}"
    # Wait for honeypot lifetime (in MINUTES, not hours)
    echo "[$(date)] Waiting ${HONEYPOT_LIFETIME_MINUTES} minutes for activity..."
    sleep $((HONEYPOT_LIFETIME_MINUTES * 60))  # Convert minutes to seconds
    # Check if there was any intrusion
    if check_for_intrusion "${ACTIVE_CONTAINER}"; then
        echo "[$(date)] Intrusion detected! Collecting logs..."
    else
        echo "[$(date)] No intrusion detected in this cycle."
    fi
    # Terminate active sessions
    terminate_ssh_sessions "${ACTIVE_CONTAINER}"
    # Collect logs
```

```
        collect_logs "${ACTIVE_CONTAINER}"
        # Short pause before next cycle
        echo "[$(date)] Cycle complete. Starting next cycle in 10 seconds..."
        sleep 10
    done
}
# Handle script termination
cleanup() {
    echo ""
    echo "[$(date)] Script interrupted. Collecting final logs..."
    if lxc info "${ACTIVE_CONTAINER}" &>/dev/null; then
        terminate_ssh_sessions "${ACTIVE_CONTAINER}"
        collect_logs "${ACTIVE_CONTAINER}"
        lxc stop "${ACTIVE_CONTAINER}" --force
    fi
    echo "[$(date)] Cleanup complete. Exiting."
    exit 0
}
trap cleanup SIGINT SIGTERM
# Check if running as root
if [ "$EUID" -ne 0 ]; then
    echo "This script must be run as root (use sudo)"
    exit 1
```

```
fi
# Verify snapshots exist
echo "Verifying snapshots..."
for treatment in "${TREATMENTS[@]}"; do
    container_name=$(echo "$treatment" | cut -d: -f1)
    snapshot_name=$(echo "$treatment" | cut -d: -f2)
    if ! lxc info "${container_name}" | grep -q "${snapshot_name}"; then
        echo "ERROR: Snapshot ${snapshot_name} not found for ${container_name}"
        echo "Please create snapshots before running this script."
        exit 1
    Fi
done
echo "All snapshots verified!"
# Start recycling loop
recycle_loop
```

References

Abewa, Y. T., & Zemene, S. (2024). Dynamic interactive honeypot for web application security. *International Journal of Wireless and Microwave Technologies, 14*(6), 1-14. https://doi.org/10.5815/ijwmt.2024.06.01

Qurbonaliyeva, D., & Abduraxmanova, G. (2025, July 02). *Analysis of Methods of Attracting Attackers in the Honeypot. ICFNDS '24: Proceedings of the 8th International Conference on Future Networks & Distributed Systems*, 897–904. https://doi.org/10.1145/3726122.3726253

Savage, E. (2025). *Profiling attacker behavior in honeypot deception*. https://doi.org/10.62791/20443

Schneider, D., Fraunholz, D., & Krohmer, D. (2021, January 6). *A qualitative empirical analysis of human Post-Exploitation behavior*. arXiv.org. https://arxiv.org/abs/2101.02102

Shah, N., Ho, G., Schweighauser, M., Afifi, M. H., Cidon, A., & Wagner, D. (2020, July 28). *A Large-Scale analysis of attacker activity in compromised enterprise accounts*. arXiv.org. https://arxiv.org/abs/2007.14030